RL-ing Nanochat

CSC490 Assignment 4

Janis Joplin (1009715051) • Louis Ryan Tan (1009713561)
Maverick Luke (1009714855) • Razan Ahsan Rifandi (1009562108)

January - April 2026

1 GRPO and RL Review

Nanochat's RL implementation takes a deliberately minimalist approach to policy optimization. It draws inspiration from GRPO [8], which itself was designed to simplify PPO by removing the need for a separate critic network and instead estimating advantages by sampling a group of outputs per prompt. However, nanochat goes much further in stripping things down. The code removes the KL divergence penalty and reference model entirely, meaning there is no mechanism to explicitly prevent the policy from drifting far from its starting point. It also drops the importance sampling ratio and clipping that GRPO inherits from PPO. This is justified because nanochat generates fresh rollouts at every training step, making it fully on-policy, so the ratio would always be 1 anyway. The advantage calculation is simplified from z-score normalization ($\frac{r-\mu}{\sigma}$) to just subtracting the mean reward ($r - \mu$). Normalization is also done at the token level rather than per-sequence. The result, as Karpathy acknowledges in the code comments, is something much closer to REINFORCE than to GRPO. The main surviving idea is the "group relative" sampling of multiple completions per question. These choices make sense given nanochat's context. At the scale of a $\sim$ 561M parameter model training on GSM8k with a binary correct/incorrect reward, the full GRPO machinery would add complexity without meaningful benefit. The conditions that make trust regions and KL penalties important, such as large models, noisy reward signals, and long training horizons, simply don't apply here.

2 SFT & Midtraining

We will use a baseline Nanochat version that is pretrained with almost the same configuration as Nanochat with the exception of using Grouped Query Attention (GQA) with a KV-head to Q-head ratio of 0.5 instead of Multi-Head Attention(MHA). This baseline is chosen because it is the model available to us without retraining an additional model. Previous experiment on a smaller scale shown similar performance, so this model will suffice. We performed supervised fine-tuning (SFT) on the baseline model; one using the default Nanochat SFT dataset, and an augmented SFT dataset.

Nanochat's SFT training dataset consists of SmolTalk [3], MMLU [2], GSM8K [1], and some small custom datasets: SimpleSpelling, SpellingBee, and synthetic identity conversations. MMLU and GSM8K are repeated 3 and 4 times respectively, resulting in a dataset of 1M rows. These datasets mainly consists of conversations and contains reasoning. To augment the downstream RL task, we added datasets containing general math questions: ASDiv [5], SVAMP [6], MAWPS [4], and MetaMathQA [9]. These datasets are chosen to improve performance on grade school math questions. Due to their small sizes, ASDiv, SVAMP, and MAWPS are replicated 4 times, contributing 25K rows and we sample 80K rows from MetaMathQA. These additional dataset increases the number rows by 10%. The following shows the evaluation benchmarks for each model. We will refer to the model trained on the original dataset `sft-default` and the model trained with the additional dataset `sft-augment`. The training logs for both SFT runs can be found here.

Table 1: Evaluation benchmarks across baseline and SFT variants

Model	ARC-Easy	ARC-Challenge	MMLU	GSM8K	HumanEval	SpellingBee	ChatCORE
`baseline`	0.2508	0.2278	0.2296	0.0000	0.0000	0.0000	-0.0093
`sft-default`	0.6338	0.4949	0.3735	0.0910	0.0610	0.9922	0.3578
`sft-augment`	0.6284	0.4966	0.3720	0.1562	0.0427	0.9883	0.3638

The pretrained model performed poorly on most benchmarks, achieving 0% accuracy on both GSM8K, HumanEval, and SpellingBee. SFT signifincantly improved the performance of the model across all benchmarks.

Notably, `sft-default` showed an improvement from 25.1% to 63.4% in ARC-Easy, and 22.8% to 49.5% in ARC-Challenge, and 0% to 99.2% in SpellingBee.This indicates that SFT strongly improves the model's ability to follow instructions and perform reasoning tasks. SFT training provides examples of desired outputs, teaching the model to structure answers and reason through problems. This explains the significant improvement across conversational and reasoning benchmarks.

Meanwhile, `sft-augment` showed a marginal improvement for GSM8K (9.1% to 15.6%). This indicates that the additional math datasets were effective in improving math capabilities. While the augmented dataset improves GSM8K performance, it slightly reduces results on ARC-Easy and HumanEval. This may be due to the increased proportion of math-focused training examples biasing the model toward mathematical reasoning tasks.

The following commands were used for perform SFT.

```
modal run nanochat_modal.py::stage_sft
  --wandb-run math-ablation-v2
  --model-tag d24-relu2-kvr2
  --output-tag math-ablation-v2
```

3 Replicating RL Run with Additional Analysis

3.1 RL Training Setup

For Part 3, we decided to conduct RL training on top of the model obtained in Part 2 (pretrained nanochat + GQA, followed by SFT and midtraining) rather than from the base pretrained checkpoint. In the nanochat repository discussion thread, the RL example run on GSM8K achieves only around 6% accuracy even when SFT has already been applied before RL. Given this empirical observation, we expect that if RL were applied directly to the base pretrained model without SFT, performance would likely be even lower because the model would not yet exhibit strong instruction-following or reasoning behavior, particularly for mathematical reasoning tasks such as GSM8K.

RL training is performed using the original nanochat RL implementation (pipeline `scripts.chat_rl`) through the Modal training script provided in the course tutorial by TA Angela Sha [7]. Starting with the model from Part 2 allows us to reuse the previously trained checkpoint and focus the RL stage on improving reasoning behavior on GSM8K, while also preserving Modal compute credits for the additional reward experiments required in Part 4.

Specifically, RL was started from model `math-ablation-v2` (the one from Part 2) at step 503, with 1 epoch, 16 examples per step, 16 samples per example, `max_new_tokens=256`, `temperature=1.0`, `top_k=50`, and evaluation every 60 steps. The CLI commands we used to train and evaluate can be seen below.

```
modal run --detach nanochat_modal.py::stage_rl
  --wandb-run a4-p3-rl
  --model-tag math-ablation-v2
  --model-step 503
```

For evaluation, we use the same setup as in Part 2 for consistency. However, we expect the post-RL model to perform worse on some non-GSM8K tasks, since the RL stage is optimized only on GSM8K. Therefore, our main focus in this section is the model's GSM8K performance and how it compares with the original RL run described in the discussion thread.

3.2 Training Curves and Comparison with the Original Run

The full run took approximately 2 hours and 51 minutes. The corresponding Weights & Biases run (including GSM8K eval results) can be found at https://api.wandb.ai/links/contribnow-none/tnobmmtz.

Table 2 summarizes the post-RL evaluation results using the same evaluation setup as in Part 2. As expected, the most important metric in this stage is GSM8K, since the RL training is performed specifically on GSM8K rather than on a broader multitask mixture. While the model is still evaluated on other benchmarks for reference, our main goal here is to compare the GSM8K-focused RL behavior against the original nanochat run.

Task	RL Score
ARC-Easy	0.6275
ARC-Challenge	0.4855
MMLU	0.3679
GSM8K	0.3487
HumanEval	0.0854
SpellingBee	0.0195
ChatCORE	0.2380

Table 2: Post-RL evaluation results using the same evaluation setup as Part 2.

Table 2 summarizes the final post-RL checkpoint, but it does not show how performance evolved during training. To study the training dynamics more directly, we compare the reward and evaluation curves from our run with those from the original nanochat RL example. This allows us to check whether our run follows a similar trend, even if the absolute values differ. The reward and eval curves for both our training run (taken from the previous link for comparison purpose) and the original run can be seen below.

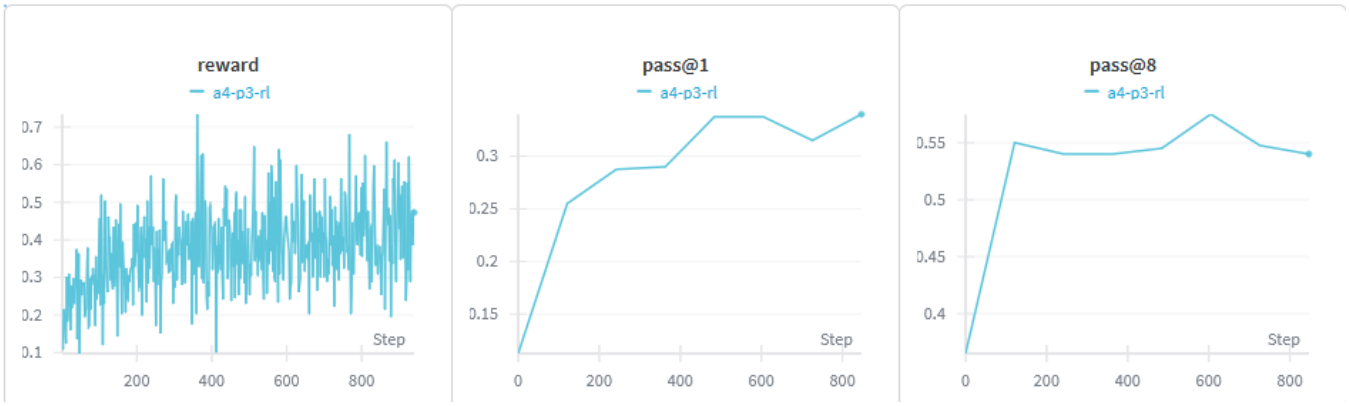

Figure 1: Our RL curves

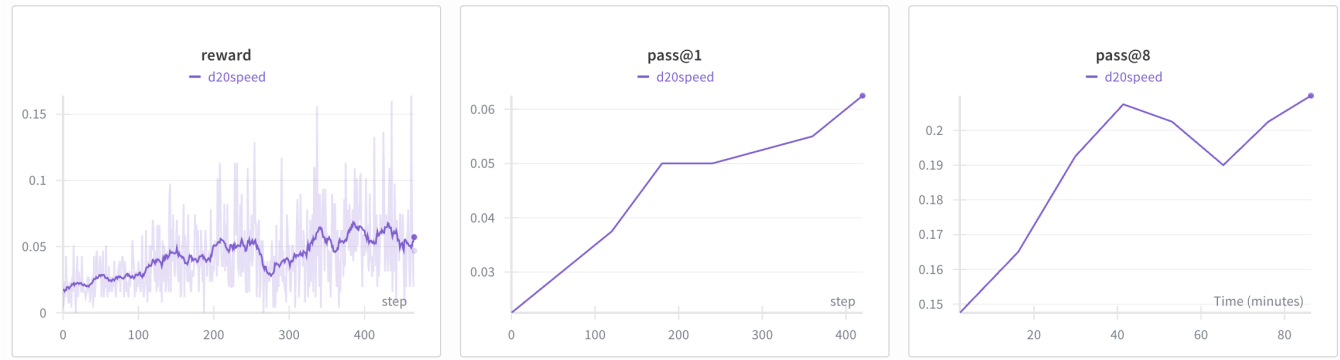

Figure 2: Original RL curves

Figures 1 and 2 compare our training curves against the original run shown in the nanochat repository discussion. In both runs, the reward increases overall despite substantial noise, which is expected since the policy samples multiple candidate solutions on each batch. In our run, the reward rises from roughly 0.1–0.2 early in training to around 0.3–0.5 later on, while `pass@1` improves from about 0.11 to about 0.34 and `pass@8` improves from about 0.37 to about 0.54. The final post-RL evaluation also reports a GSM8K accuracy of 0.3487. Qualitatively, this matches the trend in the original run, where reward, `pass@1`, and `pass@8` also increase over time. However, our curves reach much higher values than the original example, whose `pass@1` ends around 0.06 and whose `pass@8` ends around 0.21. The most likely reason is that our RL stage starts from the stronger Part 2 checkpoint, which includes our own midtraining and SFT improvements. Another likely factor is that our run lasted longer than the approximately 1.5

hour example reported by the author. Overall, although the absolute values are different, the qualitative behavior is consistent with the original nanochat RL run: reward rises during training, pass@k improves, and the gap between `pass@8` and `pass@1` suggests that the model often contains partially correct reasoning but does not yet reliably produce the correct answer on its first sample.

4 Introducing a More Complex Rewards System

The original reward system for the RL environment is a binary reward system, which assigns 1 if the model's final answer exactly matches the ground truth and 0 otherwise. This reward system is simple, interpretable, and widely used in LLM RL training, it provides no gradient signal for near-correct answers.

As we want the LLM to learn arithmetics and mathematical reasoning, not just patterns from training data, we introduce the relative correctness reward for Reward A. This reward replaces the binary correct/incorrect signal with a continuous score based on how close the predicted answer is to the ground truth.

Furthermore, the baseline reward system gives no credit for correct reasoning steps that happen to produce a wrong final number, or punishing an output with the correct final number but with incorrect reasoning step. Motivated by this, we added a reasoning step validity bonus as Reward B that rewards the model for producing correct intermediate calculation steps in tool calls present in the training data and model output.

For both Reward A and Reward B, we are motivated by the implementations of RL reward system that are widely used in literature. The main debate in papers are ORM (Outcome Reward Model) or PRM (Process Reward Model). Research finds that even though learned PRMs have strong inference-time performance, they may only bring marginal improvements or even hurt RL training, producing worse results than using simple success rewards alone, and training collapse easily occurs when PRM serves as reward shaping on top of success rewards gao2024designing. Because of this, we introduce an alternative ORM (Reward A) that is continuous instead of binary, and a PRM (Reward B) that is not present in the baseline.

4.1 Reward A: Relative Correctness Reward

Given a predicted answer $\hat{y}$ and reference answer y, the reward is defined as:

$$r_{relative}(\hat{y}, y) = \max\left(0, 1 - \frac{|\hat{y} - y|}{|y + \varepsilon|}\right)$$

where $\varepsilon = 10^{-8}$.

4.2 Reward B: Reasoning Step Bonus

Let S denote the set of tool calls found in the response and let $v(s)$ be an indicator that step $s \in S$ is arithmetically valid (expression is equal to the result). The step bonus is:

$$b_{step} = \min\left(0.3, 0.05 \cdot \sum_{s \in S} v(s)\right)$$

The combined reward is then:

$$r_{reasoning}(\hat{y}, y) = \min\left(1.0, 0.7 \cdot \mathbf{1}[\hat{y} = y] + b_{step}\right)$$

4.3 Training Results and Comparison with Baseline

Table 3: Summary of RL Run Results Across All Benchmarks

Run	**ARC-Easy**	**ARC-Challenge**	**MMLU**	**GSM8K**	**HumanEval**	**SpellingBee**	**ChatCORE**
RL Base	0.6275	0.4855	0.3679	0.3487	0.0854	0.0195	0.2380
RL Reward A	0.6309	0.4983	0.3710	0.3184	0.0610	0.6172	0.3328
RL Reward B	0.6330	0.4957	0.3696	0.2077	0.0305	0.5664	0.3004
RL Reward A+B	0.6296	0.4932	0.3673	0.2077	-	-	-

In the SpellingBee benchmark, the baseline scores near zero (0.0195), while Reward A and B reach 0.617 and 0.566 respectively, suggesting the reward signal was directly targeting this task. However, the tradeoff shows up clearly in GSM8K and HumanEval, which drop substantially under Reward B and A+B, consistent with the model over-specializing at the expense of math and coding ability. General benchmarks (ARC, MMLU, ChatCORE) are largely unaffected. Reward A comes out ahead overall, preserving more of the base model's capabilities while still achieving strong SpellingBee gains.

The training and eval curves can be found at https://api.wandb.ai/links/contribnow-none/h6lr20x1.

References

[1] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukas Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.

[2] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2020.

[3] Hugging FaceTB. smol-smoltalk Dataset Card, 2024.

[4] Rik Koncel-Kedziorski, Subhro Roy, Aida Amini, Nate Kushman, Peter Jansen, and Hannaneh Hajishirzi. MAWPS: A math word problem repository. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1152–1157. Association for Computational Linguistics, 2016.

[5] Shen-Yun Miao, Chao-Chun Liang, and Keh Yih Su. ASDiv: A diverse corpus for evaluating and developing english math word problem solvers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8332–8344. Association for Computational Linguistics, 2020.

[6] Aaryan Patel, Satwik Bhattamishra, and Navin Goyal. SVAMP: 5K math word problems with reasoning gap to arithmetic word problem solvers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 373–378. Association for Computational Linguistics, 2021.

[7] Angela Sha. Nanochat Modal Training Script for GRPO/PPO Tutorial, 2026.

[8] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.

[9] Longhui Yu, Weisen Liu, Xiang Wang, Yizhong Wang, Lingpeng Kong, and Yu Su. MetaMath: Bootstrap your own mathematical questions for large language models, 2023.